Análise e Desenvolvimento de Sistemas

Disciplina: CODING

Professora: Diego Abreu

Aluno: Giovani Amauri Frazão Brelaz

Matrícula: 04206364

Turno: Manhã

Aula 5 Parte 4 Funções Compostas e Exercícios Guiados.docx

Atividade 1 - Função booleana: par ou ímpar

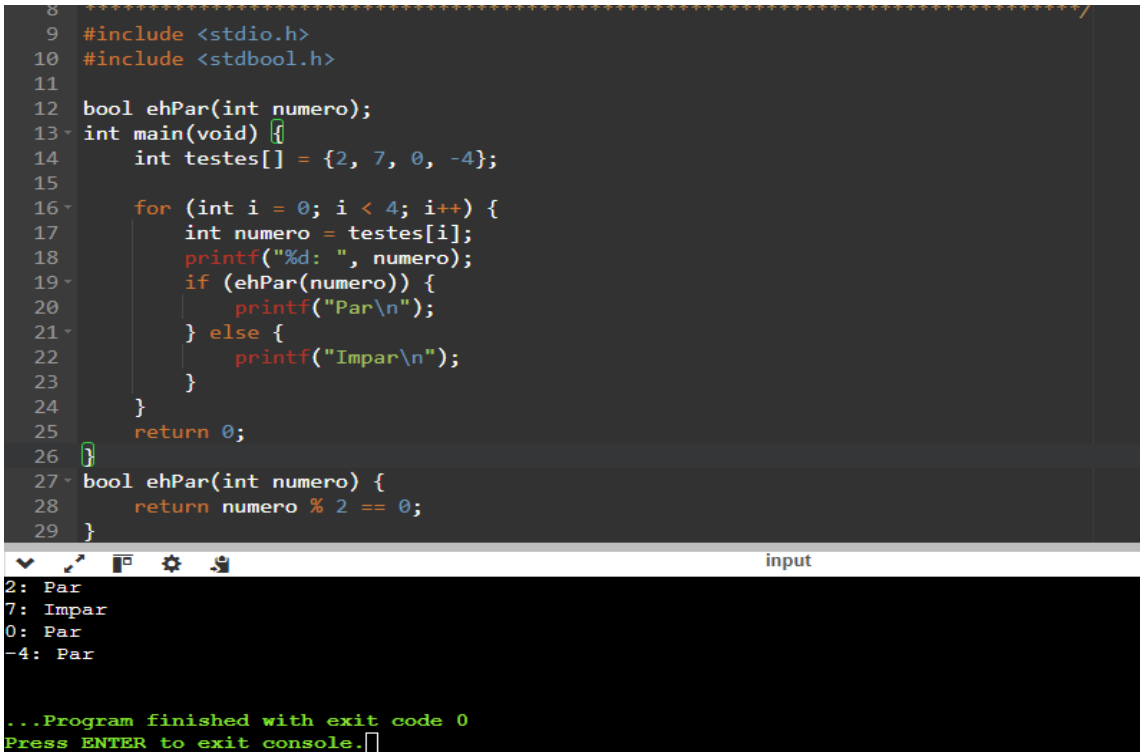
```
#include <stdbool.h>

bool ehPar(int numero) {
return numero % 2 == 0;
}
```

No main:

```
if (ehPar(numero)) {
printf("Par\n");
} else {
printf("Impar\n");
}
```

Teste com 2, 7, 0 e -4. Observe como o nome ehPar aproxima o código da linguagem do problema.



```c
#include <stdio.h>
#include <stdbool.h>

bool ehPar(int numero);
int main(void) {
    int testes[] = {2, 7, 0, -4};

    for (int i = 0; i < 4; i++) {
        int numero = testes[i];
        printf("%d: ", numero);
        if (ehPar(numero)) {
            printf("Par\n");
        } else {
            printf("Impar\n");
        }
    }
    return 0;
}
bool ehPar(int numero) {
    return numero % 2 == 0;
}
```

```
input
2: Par
7: Impar
0: Par
-4: Par


...Program finished with exit code 0
Press ENTER to exit console.
```

Atividade 2 - Nomeando uma regra

```
bool podeEntrar(int idade, bool bloqueado) {
return idade >= 18 && !bloqueado;
}
```

1. Leia idade e estado de bloqueio no main.
2. Use podeEntrar() dentro de um if.
3. Acrescente uma terceira regra: possuir ingresso.

```c
#include <stdio.h>
#include <stdbool.h>

bool podeEntrar(int idade, bool bloqueado, bool temIngresso);

int main(void) {
    int idade;
    int bloqueadoInt, ingressoInt;

    printf("Digite a idade: ");
    scanf("%d", &idade);

    printf("Esta bloqueado? (1 = sim, 0 = nao): ");
    scanf("%d", &bloqueadoInt);

    printf("Possui ingresso? (1 = sim, 0 = nao): ");
    scanf("%d", &ingressoInt);

    bool bloqueado = bloqueadoInt == 1;
    bool temIngresso = ingressoInt == 1;

    if (podeEntrar(idade, bloqueado, temIngresso)) {
        printf("Pode entrar!\n");
    } else {
        printf("Nao pode entrar.\n");
    }

    return 0;
}
```

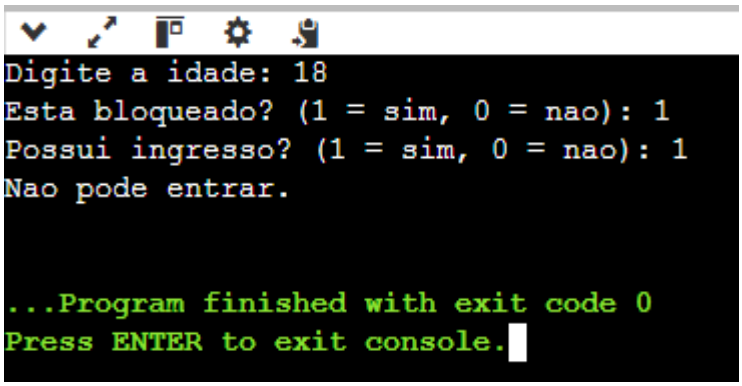
```c
bool podeEntrar(int idade, bool bloqueado, bool temIngresso) {
    return idade >= 18 && !bloqueado && temIngresso;
}
```

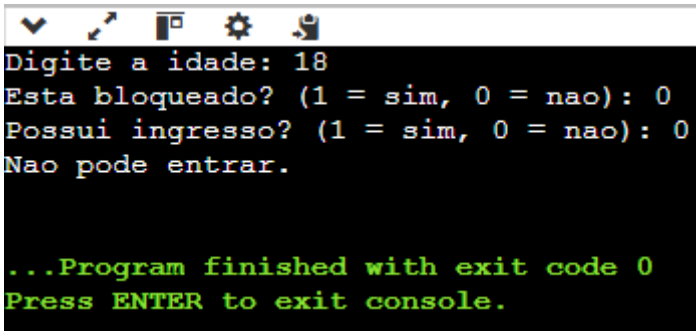
```
Digite a idade: 18
Esta bloqueado? (1 = sim, 0 = nao): 0
Possui ingresso? (1 = sim, 0 = nao): 1
Pode entrar!

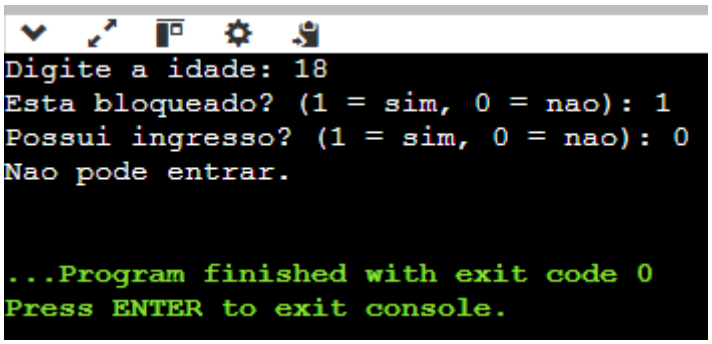

...Program finished with exit code 0
Press ENTER to exit console.
```

```
Digite a idade: 17
Esta bloqueado? (1 = sim, 0 = nao): 0
Possui ingresso? (1 = sim, 0 = nao): 1
Nao pode entrar.


...Program finished with exit code 0
Press ENTER to exit console.
```


Digite a idade: 18
Esta bloqueado? (1 = sim, 0 = nao): 1
Possui ingresso? (1 = sim, 0 = nao): 1
Nao pode entrar.
...Program finished with exit code 0
Press ENTER to exit console.


Digite a idade: 18
Esta bloqueado? (1 = sim, 0 = nao): 0
Possui ingresso? (1 = sim, 0 = nao): 0
Nao pode entrar.
...Program finished with exit code 0
Press ENTER to exit console.


Digite a idade: 18
Esta bloqueado? (1 = sim, 0 = nao): 1
Possui ingresso? (1 = sim, 0 = nao): 0
Nao pode entrar.
...Program finished with exit code 0
Press ENTER to exit console.

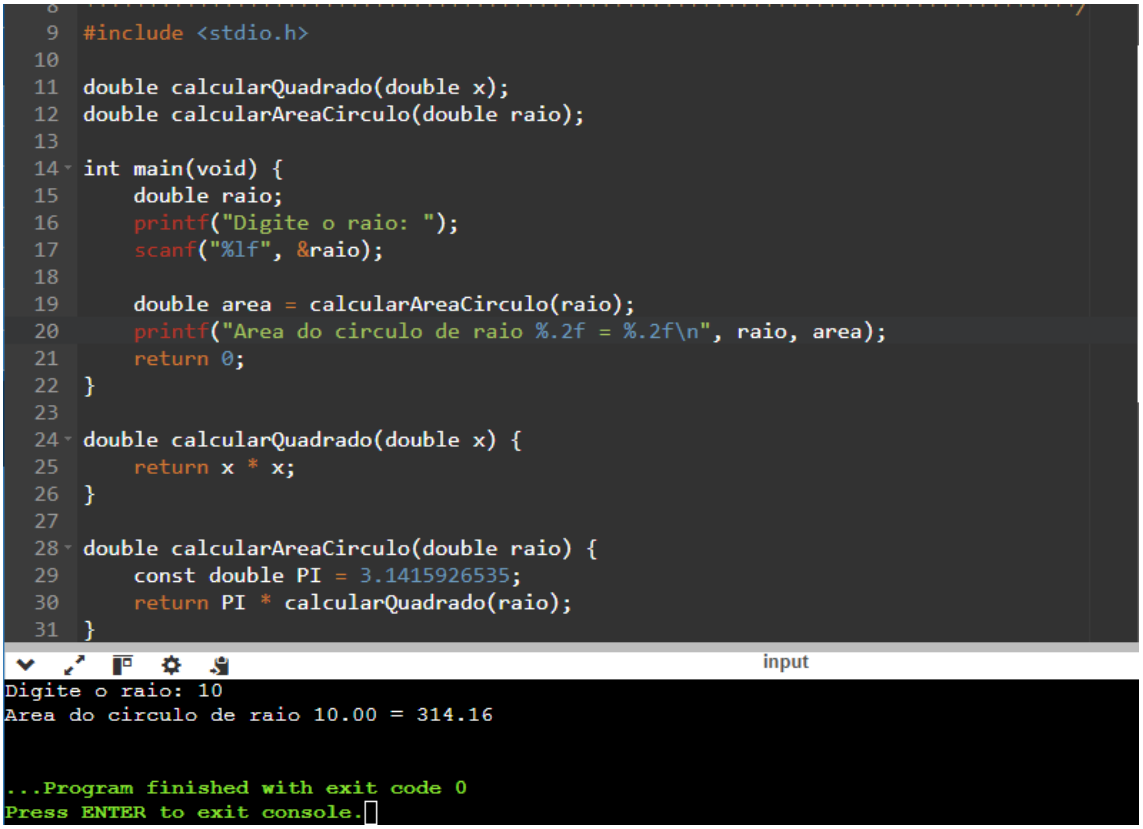
Atividade 3 - Funções chamando funções

```
double calcularQuadrado(double x) {
return x * x;
}

double calcularAreaCirculo(double raio) {
const double PI = 3.1415926535;
return PI * calcularQuadrado(raio);
}
```

Visualize o fluxo: main() chama calcularAreaCirculo(), que chama calcularQuadrado(). Quando a função interna termina, a execução retorna ao ponto anterior.

Pilha de chamadas: nesta aula basta entender conceitualmente que cada chamada suspende temporariamente a função atual e retorna a ela quando termina.



Atividade 4 - Retorno antecipado

bool notaValida(double nota) {
if (nota < 0 || nota > 10) {
return false;
}
return true;
}

Crie também uma função int maior(int a, int b) usando retorno antecipado.

```c
#include <stdio.h>
#include <stdbool.h>

bool notaValida(double nota);
int maior(int a, int b);

int main(void) {
    printf("Nota -1: %s\n", notaValida(-1) ? "Valida" : "Invalida");
    printf("Nota 7.5: %s\n", notaValida(7.5) ? "Valida" : "Invalida");
    printf("Nota 11: %s\n", notaValida(11) ? "Valida" : "Invalida");

    printf("Maior entre 10 e 20: %d\n", maior(10, 20));
    printf("Maior entre 30 e 5: %d\n", maior(30, 5));
    printf("Maior entre 7 e 7: %d\n", maior(7, 7));
    return 0;
}

bool notaValida(double nota) {
    if (nota < 0 || nota > 10) {
        return false;
    }
    return true;
}

int maior(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
```

```
Nota -1: Invalida
Nota 7.5: Valida
Nota 11: Invalida
Maior entre 10 e 20: 20
Maior entre 30 e 5: 30
Maior entre 7 e 7: 7


...Program finished with exit code 0
Press ENTER to exit console.
```

Atividade 5 - Leitura e validação dentro de função

double lerNotaValida(void) {
double nota;
do {
printf("Digite uma nota entre 0 e 10: ");
scanf("%lf", &nota);
} while (nota < 0 || nota > 10);
return nota;
}

Use a função três vezes e calcule a média com calcularMedia(). Aqui aparecem função, do-while, scanf, condição e retorno no mesmo programa.

```c
#include <stdio.h>

double lerNotaValida(void);
double calcularMedia(double n1, double n2, double n3);

int main(void) {

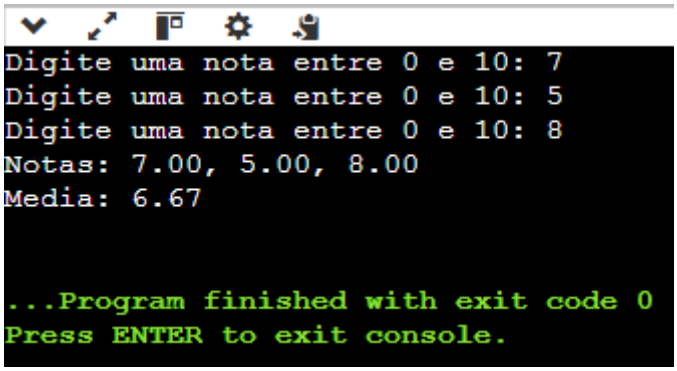
    double nota1 = lerNotaValida();
    double nota2 = lerNotaValida();
    double nota3 = lerNotaValida();

    double media = calcularMedia(nota1, nota2, nota3);

    printf("Notas: %.2f, %.2f, %.2f\n", nota1, nota2, nota3);
    printf("Media: %.2f\n", media);
    return 0;
}

double lerNotaValida(void) {
    double nota;
    do {
        printf("Digite uma nota entre 0 e 10: ");
        scanf("%lf", &nota);
    } while (nota < 0 || nota > 10); // Repete enquanto for inválida
    return nota;
}

double calcularMedia(double n1, double n2, double n3) {
    return (n1 + n2 + n3) / 3.0;
}
```



Exercícios guiados

Exercício A - Maior e menor

Implemente int maior(int a, int b) e int menor(int a, int b). Leia dois números e teste ambas.

```c
#include <stdio.h>

int maior(int a, int b);
int menor(int a, int b);

int main(void) {
    int a, b;
    printf("Digite dois numeros: ");
    scanf("%d %d", &a, &b);

    printf("Maior: %d\n", maior(a, b));
    printf("Menor: %d\n", menor(a, b));
    return 0;
}

int maior(int a, int b) {
    if (a > b) return a;
    return b;
}

int menor(int a, int b) {
    if (a < b) return a;
    return b;
}
```

```
Digite dois numeros: 10 5
Maior: 10
Menor: 5


...Program finished with exit code 0
Press ENTER to exit console.
```

Exercício B - Conversões

Implemente celsiusParaFahrenheit(), fahrenheitParaCelsius() e kmParaMilhas().

```c
#include <stdio.h>

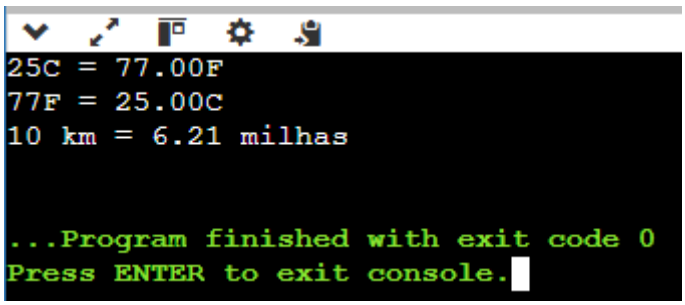
double celsiusParaFahrenheit(double c);
double fahrenheitParaCelsius(double f);
double kmParaMilhas(double km);

int main(void) {
    printf("25C = %.2fF\n", celsiusParaFahrenheit(25));
    printf("77F = %.2fC\n", fahrenheitParaCelsius(77));
    printf("10 km = %.2f milhas\n", kmParaMilhas(10));
    return 0;
}

double celsiusParaFahrenheit(double c) {
    return (c * 9.0 / 5.0) + 32;
}

double fahrenheitParaCelsius(double f) {
    return (f - 32) * 5.0 / 9.0;
}

double kmParaMilhas(double km) {
    return km * 0.621371;
}
```



Exercício C - Sistema acadêmico

double calcularMedia(double n1, double n2, double n3);
bool aprovado(double media, double frequencia);
void mostrarResultado(double media, double frequencia);

Regra: média >= 7 e frequência >= 75. Separe claramente cálculo, regra e apresentação.

```c
/*****************************************************************************/
#include <stdio.h>
#include <stdbool.h>

double calcularMedia(double n1, double n2, double n3);
bool aprovado(double media, double frequencia);
void mostrarResultado(double media, double frequencia);

int main(void) {
    double n1, n2, n3, freq;
    printf("Digite 3 notas e a frequencia: ");
    scanf("%lf %lf %lf %lf", &n1, &n2, &n3, &freq);

    double media = calcularMedia(n1, n2, n3);
    mostrarResultado(media, freq);
    return 0;
}

double calcularMedia(double n1, double n2, double n3) {
    return (n1 + n2 + n3) / 3.0;
}

bool aprovado(double media, double frequencia) {
    return media >= 7 && frequencia >= 75;
}

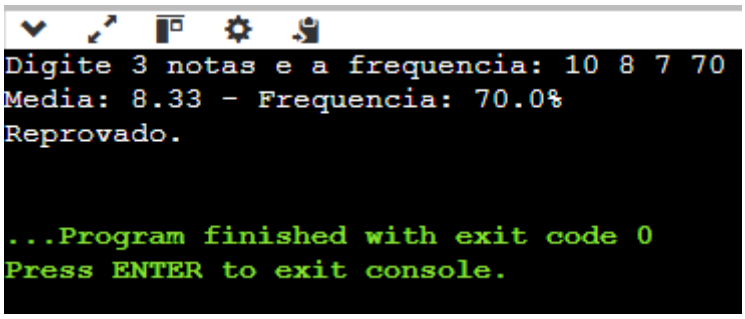
void mostrarResultado(double media, double frequencia) {
    printf("Media: %.2f - Frequencia: %.1f%%\n", media, frequencia);
    if (aprovado(media, frequencia)) {
        printf("Aprovado!\n");
    } else {
        printf("Reprovado.\n");
    }
}
```

```
Digite 3 notas e a frequencia: 10 8 9 90
Media: 9.00 - Frequencia: 90.0%
Aprovado!


...Program finished with exit code 0
Press ENTER to exit console.
```

```
Digite 3 notas e a frequencia: 5 6 8 80
Media: 6.33 - Frequencia: 80.0%
Reprovado.


...Program finished with exit code 0
Press ENTER to exit console.
```



Exercício D - Tabuada como função

void mostrarTabuada(int numero) {
for (int i = 1; i <= 10; i++) {
printf("%d x %d = %d\n", numero, i, numero * i);
}
}

Desafio: implemente void mostrarTabuadas(int inicio, int fim) e reutilize mostrarTabuada().

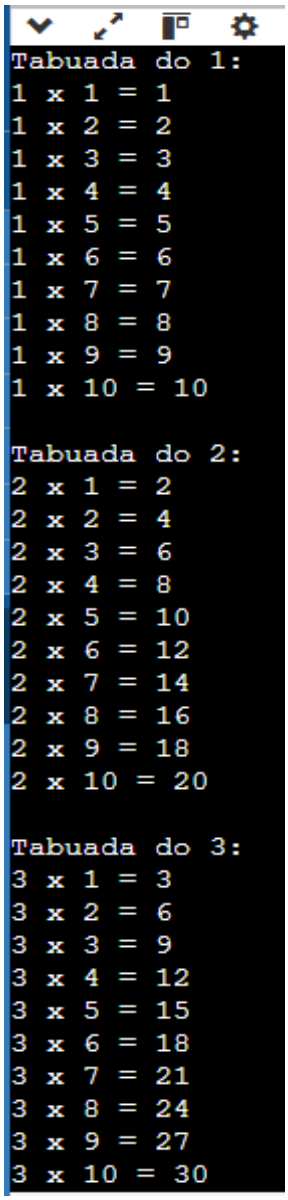
```c
#include <stdio.h>

void mostrarTabuada(int numero);
void mostrarTabuadas(int inicio, int fim);

int main(void) {
    mostrarTabuadas(1, 3);
    return 0;
}

void mostrarTabuada(int numero) {
    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", numero, i, numero * i);
    }
    printf("\n");
}

// Desafio: reutiliza a função de cima
void mostrarTabuadas(int inicio, int fim) {
    for (int n = inicio; n <= fim; n++) {
        printf("Tabuada do %d:\n", n);
        mostrarTabuada(n);
    }
}
```



4. Qual vantagem existe em encapsular numero % 2 == 0 dentro de ehPar()?
Resposta: O código fica mais legível e mais próximo da linguagem do problema. E se a regra mudar, só precisa alterar em um lugar, e a função pode ser reutilizada em qualquer parte do programa.

5. O que acontece com a execução quando uma função chama outra?
Resposta: A função atual é temporariamente suspensa e a execução vai para a função chamada. Quando a função chamada termina, ela retorna o valor para o ponto exato onde foi chamada, e a função anterior continua de onde parou.

6. Por que lerNotaValida() é mais reutilizável do que repetir o do-while em vários pontos?
Resposta: Isso evita repetição de código, facilita a manutenção e reduz o risco de erro, pois qualquer correção é feita uma única vez dentro da função.